CÔNG TY TNHH DEVLINUX
ĐÀO TẠO LẬP TRÌNH EMBEDDED CHUYÊN SÂU
Địa chỉ: LK05B No10 Khu Đồng Dưa, Quận Hà Đông,TP Hà Nội
ĐT: **0877123348** mail: **devlinux.edu@gmail.com**

Bài tập giữa kỳ

Sensor Gateway – Hệ thống giám sát cảm biến smart home

Kiến trúc tổng thể, trình tự triển khai và căn cứ lựa chọn kỹ thuật

Liêm chính học thuật

Đây là bài tập ***cá nhân***! Bạn chỉ được phép nộp ***bài làm của chính mình***. *Bạn có thể thảo luận đề bài với người khác nhưng không được chia sẻ bài làm hay sử dụng (một phần) lời giải của người khác.*

Sử dụng công cụ trí tuệ nhân tạo (AI)

- **Hiểu và tự bảo vệ được.** Mọi dòng code và mọi hình vẽ trong bài là của bạn, dù do ai hay công cụ nào gợi ý. Trong buổi bảo vệ, bạn phải giải thích được từng phần và trả lời được các câu hỏi về lập trình C, system call của Linux, đồng bộ hóa, giao tiếp giữa các tiến trình và công cụ build. Phần nào bạn không giải thích được thì không được tính, bất kể nguồn gốc. Bạn phải tự phản biện được lựa chọn của mình, nên nội dung phản biện đó phải do bạn viết.
- **Kiểm chứng.** Code do AI gợi ý thường chạy được nhưng có thể sai ở những chỗ khó thấy (data race, deadlock, rò rỉ bộ nhớ, xử lý lỗi). Bạn phải tự kiểm thử và chạy valgrind và helgrind như với mọi đoạn code khác.
- **Được phép.** Nhờ AI giải thích khái niệm (mutex, fork, FIFO, shared memory...), giải thích thông báo lỗi, gợi ý hướng tiếp cận, nhận xét code bạn đã viết, hoặc chỉnh câu chữ.

1. Mục tiêu

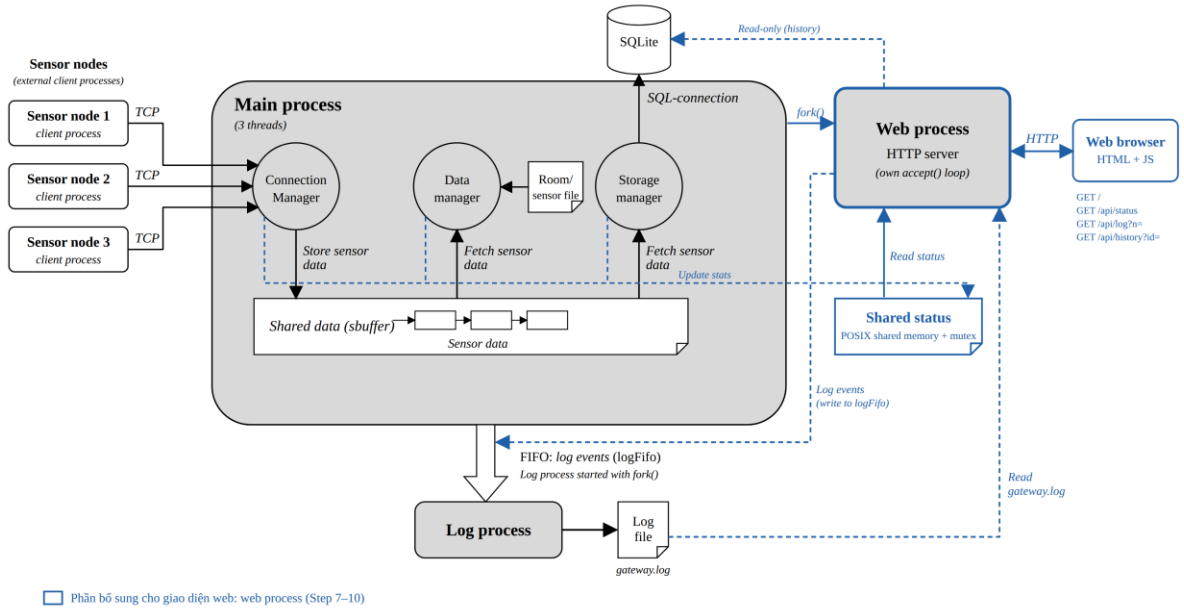
Dự án xây dựng một sensor gateway trên Linux bằng ngôn ngữ C để giám sát nhiệt độ phòng, gồm năm chức năng:

- **Thu nhận.** Tiếp nhận phép đo nhiệt độ từ nhiều sensor node (client process) qua TCP. Số lượng sensor node không cố định và có thể thay đổi theo thời gian.
- **Xử lý và cảnh báo.** Tính giá trị trung binh nhiệt độ của từng sensor, xác định trạng thái quá nóng/quá lạnh và phát hiện dữ liệu có sensor ID không hợp lệ.
- **Lưu trữ.** Ghi phép đo vào cơ sở dữ liệu SQLite; duy trì hoạt động khi mất kết nối cơ sở dữ liệu (thử lại tối đa 3 lần rồi kết thúc có kiểm soát).
- **Ghi nhật ký.** Gom các log-event của toàn hệ thống qua FIFO về một log process và ghi vào file gateway.log.
- **Hiển thị.** Cung cấp giao diện web để theo dõi hệ thống từ trình duyệt, chạy trên một web process riêng và chỉ có quyền đọc (read-only). Giao diện hiển thị bốn nhóm thông tin:
    - trạng thái từng sensor: số phòng, sensor ID, nhiệt độ mới nhất, giá trị trung binh, trạng thái (đang thu thập / bình thường / quá nóng / quá lạnh) và tình trạng kết nối;
    - trạng thái gateway: kết nối SQL, tổng số phép đo đã nhận và đã lưu, số phần tử trong sbuffer, thời gian hoạt động;
    - biểu đồ lịch sử nhiệt độ của từng sensor;
    - nhật ký gateway.log, có lọc theo nguồn (connection, data, storage, web).

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Giao diện tự làm mới ít nhất mỗi 2 giây mà không cần tải lại trang, và vẫn hiển thị đúng khi mở đồng thời nhiều tab.

2. Kiến trúc hệ thống



Hình 1: Sơ đồ khối của sensor gateway (3 process; main process chạy 3 thread)

2.1 Vai trò của từng thành phần

Thành phần	Dạng thực thi	Vai trò	Giao tiếp
Sensor node 1…N	Client process, bên ngoài gateway (mô phỏng)	Đo nhiệt độ của một phòng và gửi định kỳ gói tin gồm sensor ID, nhiệt độ và thời điểm đo. Số lượng node thay đổi theo thời gian.	TCP tới Connection manager; mỗi node một kết nối riêng
Main process	Process chính, chạy 3 thread	Chứa lõi của gateway: khởi tạo sbuffer và shared status, tạo log process và web process bằng fork(), khởi chạy ba thread và thu dọn tài nguyên khi kết thúc.	Log process (FIFO); web process (shared memory)
Connection manager	Thread	Lắng nghe TCP (cổng lấy từ tham số dòng lệnh), chấp nhận kết nối từ sensor node mới, đọc gói tin và ghi phép đo vào sbuffer; ghi log khi kết nối được mở hoặc đóng.	TCP (đầu vào); sbuffer
Data manager	Thread	Đọc sbuffer, tra bảng room–sensor (nạp từ file room–sensor khi khởi động), tính giá trị trung binh (running average) của nhiệt độ, xác định trạng thái quá nóng/quá lạnh và ghi log.	sbuffer; file room–sensor; shared status
Storage manager	Thread	Đọc sbuffer và ghi phép đo vào SQLite. Khi mất kết nối cơ sở dữ liệu, thử kết nối lại; sau 3 lần thất bại thì kết thúc gateway.	sbuffer; SQLite
sbuffer	Cấu trúc dữ liệu dùng chung trong main process	Hàng đợi phép đo duy nhất giữa ba thread. Một phần tử chỉ bị loại bỏ khi cả Data manager và Storage manager đã đọc.	Mutex + condition variable

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Thành phần	Dạng thực thi	Vai trò	Giao tiếp
Log process	Process con (fork)	Nhận log-event từ các thread và web process qua FIFO, ghi vào gateway.log theo định dạng <số thứ tự> <thời điểm> <nguồn> <thông điệp> (Mục 3.8).	FIFO logFifo; file gateway.log
Web process	Process con (fork)	HTTP server chỉ đọc: phục vụ trang web và các API JSON về trạng thái sensor, trạng thái gateway, nhật ký và lịch sử phép đo.	Shared status; SQLite (chỉ đọc); gateway.log; FIFO (ghi log)
Shared status	POSIX shared memory + mutex liên process	Bảng trạng thái hiện hành: bộ đếm, bảng sensor, trạng thái kết nối SQL, kích thước sbuffer, thời gian hoạt động (uptime).	Ba thread ghi; web process đọc
SQLite DB	File cơ sở dữ liệu	Lưu trữ toàn bộ phép đo.	Storage manager ghi; web process đọc
File room–sensor	File cấu hình (văn bản), chỉ đọc	Danh mục cấu hình các sensor hợp lệ: mỗi dòng khai báo một sensor gồm phòng, sensor ID và loại sensor. Data manager nạp một lần khi khởi động để (1) xác thực sensor ID và loại sensor của dữ liệu nhận được, (2) gắn số phòng cho từng sensor khi ghi log và hiển thị. Không chứa phép đo và không thay đổi khi chạy.	Data manager đọc một lần khi khởi động

2.2 Đường đi của dữ liệu

1. Sensor node gửi dữ liệu (sensor ID, giá trị, thời điểm đo) qua kết nối TCP riêng, theo giao thức của loại sensor.
2. Connection manager nhận byte từ socket; protocol adapter tương ứng giải mã thành sensor_data_t; nếu đó là frame hợp lệ đầu tiên của node thì ghi log mở kết nối; sau đó ghi vào sbuffer.
3. Data manager đọc phần tử từ sbuffer, xử lý theo loại sensor (giá trị trung binh, so ngưỡng hoặc ghi nhận sự kiện), ghi log khi có cảnh báo và cập nhật shared status.
4. Storage manager đọc cùng phép đo từ sbuffer và ghi vào SQLite. Khi cả hai thread đã đọc, phần tử được loại khỏi sbuffer.
5. Các sự kiện đáng chú ý được gửi qua FIFO tới log process, nơi ghi vào gateway.log.
6. Trình duyệt gửi yêu cầu tới web process; web process lấy trạng thái từ shared status, lịch sử từ SQLite, nhật ký từ gateway.log và trả kết quả.

2.3 Khởi chạy và tài nguyên

- **Khởi chạy:** ./server <cổng gốc P> <cổng HTTP>, ví dụ ./server 1234 8080. Gateway lắng nghe bốn cổng TCP từ P đến P+3 cho sensor node (mỗi cổng một giao thức, xem Mục 3) và cổng HTTP cho trình duyệt.
- **Tài nguyên:** FIFO logFifo; file nhật ký gateway.log; file ánh xạ room–sensor (danh mục cấu hình các sensor hợp lệ: phòng, sensor ID, loại sensor); file SQLite; vùng shared memory (ví dụ /gatewayStatus); thư mục web/ chứa trang giao diện.

3. Đặc tả sensor và giao thức (cung cấp sẵn)

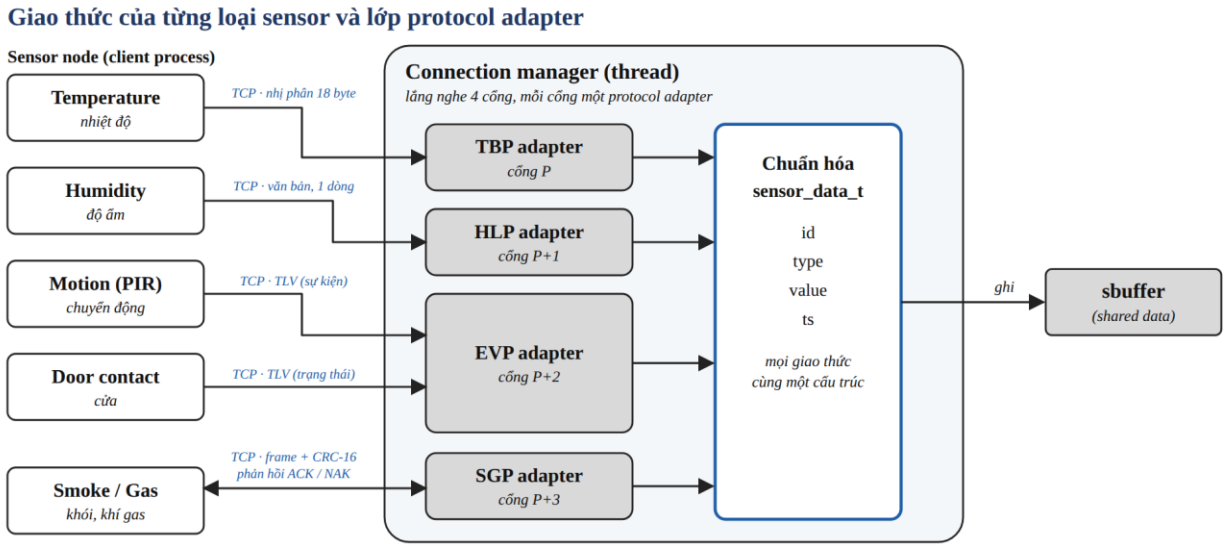
3.1 Nguyên tắc thiết kế

- **Mỗi loại sensor có thể có giao thức riêng.** Thiết bị của các nhà sản xuất khác nhau không dùng chung định dạng. Bốn giao thức trong tài liệu (TBP, HLP, EVP, SGP) khác

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

nhau về mã hóa (nhị phân hoặc văn bản), cách phân khung (độ dài cố định, ký tự xuống dòng, TLV), thứ tự byte và cơ chế tin cậy.

- **Sự khác biệt được cô lập trong protocol adapter.** Mỗi giao thức có một adapter giải mã byte thành cấu trúc chung sensor_data_t. Từ sbuffer trở đi, Data manager, Storage manager và web process chỉ làm việc với sensor_data_t và không biết giao thức gốc.
- **Dễ mở rộng.** Thêm một loại sensor hoặc giao thức mới chỉ cần thêm một adapter và một handler xử lý, không phải sửa sbuffer, Storage manager hay giao diện.
- **Quy ước kết nối.** Mỗi sensor node giữ một kết nối TCP riêng, liên tục. Đóng kết nối nghĩa là sensor ngừng hoạt động. Sensor ID chỉ biết sau frame hợp lệ đầu tiên.



Hình 3: Giao thức của từng loại sensor và lớp protocol adapter

3.2 Cấu trúc dữ liệu chuẩn hóa

Mọi giao thức đều được giải mã về cùng một cấu trúc sensor_data_t. Đây là nội dung của mỗi phần tử trong sbuffer (cùng với hai cờ đã đọc của Step 3).

```
/* sensor_types.h - cung cấp sẵn, không sửa */
#include <stdint.h>
#include <time.h>

typedef uint16_t sensor_id_t;

typedef enum {
    SENSOR_TEMPERATURE = 1,   /* °C                          */
    SENSOR_HUMIDITY    = 2,   /* %RH                         */
    SENSOR_MOTION      = 3,   /* 1 = phát hiện chuyển động   */
    SENSOR_DOOR        = 4,   /* 0 = đóng, 1 = mở            */
    SENSOR_SMOKE_GAS   = 5    /* ppm                         */
} sensor_type_t;

typedef struct {
    sensor_id_t   id;     /* duy nhất trong hệ thống            */
    sensor_type_t type;   /* loại sensor                        */
    double        value;  /* theo đơn vị của loại sensor        */
    time_t        ts;     /* thời điểm đo, giây, UTC            */
} sensor_data_t;
```

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

3.3 Các loại sensor, ngưỡng và cách xử lý

Loại sensor	Giá trị (value) và dải hợp lệ	Xử lý tại Data manager	Ngưỡng mặc định	Cảnh báo
Temperature (type = 1)	°C, từ -40 đến 85	Giá trị trung binh của 5 phép đo gần nhất	Lạnh < 15 °C; nóng > 20 °C	Quá lạnh / quá nóng
Humidity (type = 2)	%RH, từ 0 đến 100	Giá trị trung binh của 5 phép đo gần nhất	Khô < 30 %; ẩm > 70 %	Quá khô / quá ẩm
Motion (type = 3)	1 = phát hiện chuyển động	Ghi nhận mỗi sự kiện	Không có	Phát hiện chuyển động
Door (type = 4)	0 = đóng, 1 = mở	Ghi nhận mỗi lần thay đổi trạng thái	Không có	Cửa mở / cửa đóng
Smoke/Gas (type = 5)	ppm, từ 0 đến 10000	So ngưỡng tức thời, không lấy trung bình vì cần phản ứng ngay	Báo động khi ≥ 300 ppm	Báo động khói/gas

Các ngưỡng được định nghĩa trong sensor_config.h (cung cấp sẵn); có thể đổi giá trị nhưng không đổi tên macro.

```
/* sensor_config.h - cung cấp sẵn */
#define RUN_AVG_LENGTH     5      /* số phép đo của giá trị trung binh */
#define TEMP_MIN_C         15.0
#define TEMP_MAX_C         20.0
#define HUMIDITY_MIN       30.0
#define HUMIDITY_MAX       70.0
#define GAS_ALARM_PPM      300.0
```

3.4 Giao thức của từng loại sensor

Gateway lắng nghe bốn cổng TCP liên tiếp, tính từ cổng gốc P (tham số dòng lệnh thứ nhất). Cổng mà sensor kết nối tới quyết định giao thức, nên gateway không phải đoán định dạng của dữ liệu nhận được.

Giao thức	Sensor dùng	Cổng	Phân khung	Mã hóa	Phản hồi	Lý do khác biệt
TBP	Temperature	P	Độ dài cố định, 18 byte	Nhị phân, little-endian	Không	Gói gọn nhẹ, định dạng đơn giản
HLP	Humidity	P+1	Ký tự xuống dòng (LF)	Văn bản ASCII	Không	Dễ gỡ lỗi bằng netcat; minh họa phân khung theo dòng
EVP	Motion, Door	P+2	TLV (type–length–value)	Nhị phân, big-endian	Không	Chỉ gửi khi có sự kiện; một giao thức dùng cho nhiều loại sensor
SGP	Smoke/Gas	P+3	Độ dài cố định, 11 byte, bắt đầu bằng 0x7E	Nhị phân, big-endian, CRC-16	ACK / NAK	Dữ liệu an toàn: cần phát hiện frame hỏng và xác nhận đã nhận

3.4.1 TBP – Temperature Binary Protocol (cổng P)

Frame cố định 18 byte, các trường nhiều byte theo little-endian, không có ký tự phân cách và không có phản hồi.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Offset	Kích thước	Trường	Mã hóa	Ghi chú
0	2 byte	id	uint16, little-endian	Sensor ID
2	8 byte	value	IEEE 754 double, little-endian	Nhiệt độ, °C
10	8 byte	ts	int64, little-endian	Giây, UTC

Ví dụ (id = 10, 21,5 °C, ts = 1767225600):

```
0A 00 | 00 00 00 00 00 80 35 40 | 00 B9 55 69 00 00 00 00
```

3.4.2 HLP – Humidity Line Protocol (cổng P+1)

Mỗi thông điệp là một dòng văn bản ASCII kết thúc bằng ký tự LF (0x0A), các trường cách nhau đúng một dấu cách, số thực dùng dấu chấm. Độ dài tối đa 64 byte kể cả LF; dòng dài hơn mà chưa gặp LF là lỗi. Không có phản hồi.

```
HUM <id> <value> <ts>\n

HUM 21 54.3 1767225600\n
```

3.4.3 EVP – Event TLV Protocol (cổng P+2)

Frame có dạng type (1 byte), length (1 byte), payload (length byte); các trường nhiều byte theo big-endian. Sensor chỉ gửi khi có sự kiện hoặc khi trạng thái thay đổi. Type hoặc length không hợp lệ là lỗi; không có phản hồi.

Type	Loại	Length	Payload	Kích thước frame
0x01	Motion	6	id (uint16) + ts (uint32)	8 byte
0x02	Door	7	id (uint16) + state (uint8: 0 = đóng, 1 = mở) + ts (uint32)	9 byte

Ví dụ:

```
Motion, id = 30 :  01 06 | 00 1E | 69 55 B9 00
Door,   id = 40, mở :  02 07 | 00 28 | 01 | 69 55 B9 00
```

3.4.4 SGP – Smoke/Gas Protocol (cổng P+3)

Frame cố định 11 byte, big-endian, có kiểm tra CRC-16 và phản hồi từ gateway.

Offset	Kích thước	Trường	Mã hóa	Ghi chú
0	1 byte	magic	0x7E	Đánh dấu bắt đầu frame
1	2 byte	id	uint16, big-endian	Sensor ID
3	2 byte	ppm	uint16, big-endian	Nồng độ, ppm
5	4 byte	ts	uint32, big-endian	Giây, UTC
9	2 byte	crc16	uint16, big-endian	CRC của các byte từ offset 1 đến 8

- **CRC-16/CCITT-FALSE:** đa thức 0x1021, giá trị khởi tạo 0xFFFF, không đảo bit, không XOR đầu ra. Vector kiểm tra: CRC của chuỗi ASCII “123456789” là 0x29B1.
- **Phản hồi:** gateway gửi 1 byte: ACK (0x06) nếu magic và CRC đúng, NAK (0x15) nếu sai. Sensor chưa nhận được ACK sau 1 giây hoặc nhận NAK thì gửi lại, tối đa 3 lần.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Frame trùng do gửi lại (cùng id và ts) phải được xử lý nhất quán: bỏ qua hoặc lưu cả hai, và nêu rõ lựa chọn.

Ví dụ (id = 50, 120 ppm, ts = 1767225600, CRC = 0xACF5):

```
7E | 00 32 | 00 78 | 69 55 B9 00 | AC F5
```

3.5 Giao diện protocol adapter

Giao diện dưới đây được cung cấp sẵn. Sinh viên hiện thực hàm parse() cho từng giao thức (và build_reply() cho SGP) rồi đăng ký vào bảng ADAPTERS ở Connection manager (Step 4).

```
/* protocol_adapter.h - cung cấp sẵn */
#include <stddef.h>
#include <stdint.h>
#include "sensor_types.h"

typedef enum {
    PARSE_ERROR     = -1,   /* frame hỏng: sai magic, độ dài, CRC, giá trị */
    PARSE_NEED_MORE =  0,   /* chưa đủ byte để tạo thành một frame         */
    PARSE_OK        =  1    /* đã giải mã đúng một frame                   */
} parse_result_t;

typedef struct {
    const char *name;         /* "TBP", "HLP", "EVP", "SGP"            */
    uint16_t    port_offset;  /* cổng lắng nghe = cổng gốc + port_offset  */

    /* Giải mã tối đa một frame ở đầu buf.                               */
    /*  PARSE_OK:        điền *out, *consumed = số byte đã dùng.         */
    /*  PARSE_NEED_MORE: giữ nguyên byte, chờ thêm dữ liệu.              */
    /*  PARSE_ERROR:     *consumed = số byte cần bỏ để đồng bộ lại.      */
    parse_result_t (*parse)(const uint8_t *buf, size_t len,
                            sensor_data_t *out, size_t *consumed);

    /* Tạo phản hồi gửi lại sensor; trả 0 nếu giao thức không có phản hồi. */
    size_t (*build_reply)(parse_result_t result, uint8_t *reply, size_t cap);
} protocol_adapter_t;
```

3.6 File room–sensor và bảng cơ sở dữ liệu

Vai trò của file room–sensor. File này là danh mục cấu hình các sensor hợp lệ của hệ thống, không phải dữ liệu đo. Data manager nạp file một lần khi khởi động và dùng để: (1) xác thực: sensor ID không có trong file, hoặc có nhưng loại sensor không khớp, bị coi là dữ liệu không hợp lệ (ghi log “Received sensor data with invalid sensor node ID <node-ID>”); (2) gắn phòng: tra số phòng của từng sensor để ghi log và đưa vào shared status cho giao diện web. File chỉ được đọc trong suốt thời gian chạy.

File ánh xạ room–sensor (cung cấp sẵn, một dòng cho mỗi sensor; dòng bắt đầu bằng # là chú thích). Một phòng có thể có nhiều sensor thuộc nhiều loại khác nhau:

```
# room  sensor_id  type
501     10         temperature
501     21         humidity
602     30         motion
703     40         door
804     50         smoke_gas
905     11         temperature
905     22         humidity
```

Bảng SQLite dùng chung cho mọi loại sensor, nhờ dữ liệu đã được chuẩn hóa thành sensor_data_t:

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

```
CREATE TABLE SensorData (
    id    INTEGER NOT NULL,   -- sensor ID
    type  INTEGER NOT NULL,   -- sensor_type_t
    value REAL    NOT NULL,   -- theo đơn vị của loại sensor
    ts    INTEGER NOT NULL    -- giây, UTC
);
```

3.7 Chương trình mô phỏng sensor

Chương trình sensor_node (cung cấp sẵn) mô phỏng một sensor node: kết nối tới cổng ứng với loại sensor, sinh dữ liệu hợp lý và gửi theo đúng giao thức của loại đó.

```
./sensor_node <loại> <id> <host> <cổng gốc P> <chu kỳ ms> [tùy chọn]

  loại:     temperature | humidity | motion | door | smoke_gas
  tùy chọn: --fragment   chia mỗi frame thành nhiều lần send(), có khoảng nghỉ
            --glue       gộp nhiều frame vào một lần send()
            --corrupt    thỉnh thoảng gửi frame sai (sai CRC, sai magic, sai độ dài)

  ./sensor_node temperature 10 127.0.0.1 1234 1000
  ./sensor_node smoke_gas   50 127.0.0.1 1234 2000 --corrupt
```

- Với motion và door, chu kỳ ms là khoảng thời gian trung bình giữa hai sự kiện (ngẫu nhiên), không phải chu kỳ cố định.
- Ba tùy chọn --fragment, --glue và --corrupt dùng để kiểm thử phần phân khung và kiểm tra lỗi ở Step 4; chúng mô phỏng các tình huống thực tế của luồng byte TCP và đường truyền nhiễu.

3.8 Định dạng nhật ký (gateway.log)

Log process ghi mỗi log-event thành một dòng trong gateway.log theo định dạng cố định, để việc kiểm thử và hiển thị trên giao diện web có thể phân tích tự động.

```
<seq> <thời điểm> <nguồn> <thông điệp>

seq         số thứ tự tăng dần từ 1, do log process cấp
thời điểm   YYYY-MM-DD HH:MM:SS (giờ địa phương)
nguồn       MAIN | CONN | DATA | STOR | WEB
thông điệp  văn bản ASCII, một dòng, tối đa 200 byte
```

Thông điệp gửi qua FIFO có dạng <nguồn> <thông điệp> kết thúc bằng ký tự xuống dòng, được ghi bằng một lệnh write() không vượt quá PIPE_BUF; log process bổ sung seq và thời điểm khi ghi vào file.

Nguồn	Sự kiện	Thông điệp
MAIN	Khởi động	Gateway started (TCP base port <P>, HTTP port <H>)
MAIN	Kết thúc	Gateway shutting down (<reason>)
CONN	Kết nối thành công	A sensor node with <sensorNodeID> has opened a new connection
CONN	Đóng kết nối	The sensor node with <sensorNodeID> has closed the connection
CONN	Kết nối thất bại	Connection attempt failed on <protocol> port (<reason>)
CONN	Frame hỏng	Received malformed frame on <protocol> port (<reason>)
DATA	Cảnh báo theo loại sensor	Các thông điệp nêu ở Step 5 (too cold, too hot, too dry, too humid, detected motion, door is open / closed, smoke/gas alarm)
DATA	Sensor ID không hợp lệ	Received sensor data with invalid sensor node ID <node-ID>

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Nguồn	Sự kiện	Thông điệp
STOR	Kết nối SQL	Connection to SQL server established. / Connection to SQL server lost. / Unable to connect to SQL server.
STOR	Tạo bảng	New table <name-of-table> created.
WEB	Khởi động web	Web server started on port <port>.
WEB	Yêu cầu không hợp lệ	Web server received an invalid request (status code <status>).

<reason> nhận một trong các giá trị cố định: accept error, too many connections, malformed first frame, unknown sensor, bad CRC, bad length.

Ví dụ:

```
1 2026-10-06 09:15:02 MAIN Gateway started (TCP base port 1234, HTTP port 8080)
2 2026-10-06 09:15:02 STOR Connection to SQL server established.
3 2026-10-06 09:15:02 STOR New table SensorData created.
4 2026-10-06 09:15:07 CONN A sensor node with 10 has opened a new connection
5 2026-10-06 09:15:09 CONN Connection attempt failed on SGP port (malformed first
frame)
6 2026-10-06 09:15:31 DATA The sensor node with 10 reports it's too hot (running avg
temperature = 21.4)
7 2026-10-06 09:15:40 CONN The sensor node with 10 has closed the connection
```

Phía sensor node (chương trình mô phỏng) ghi ra stdout một dòng khi kết nối thành công: <thời điểm> sensor <id> <type> connected to <host>:<port>, hoặc khi thất bại: <thời điểm> sensor <id> <type> connect failed (<strerror>), để đối chiếu với gateway.log ở phía gateway.

3.9 Cấu trúc dữ liệu của các thành phần

Các cấu trúc dưới đây được cung cấp sẵn làm khung. Sinh viên có thể bổ sung trường nhưng không đổi tên và ý nghĩa các trường đã có. Cột Step cho biết Step đầu tiên dùng đến cấu trúc đó.

Cấu trúc	Thành phần	Step
sensor_data_t, sensor_info_t	Sensor; file room–sensor	3, 5
sbuffer_node_t, sbuffer_t	sbuffer	3
connection_t, connmgr_t	Connection manager	3, 4
datamgr_sensor_t, datamgr_t	Data manager	5
storagemgr_t và bảng SensorData	Storage manager, SQLite	6
sensor_status_t, gateway_status_t	Shared status	8
gateway_t	Ngữ cảnh gốc của main process	1

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

```
/* ---- Hằng số và kiểu liệt kê dùng chung ---- */
#define RX_BUF_SIZE  256
#define MAX_CONN      64
#define MAX_SENSORS   64
typedef enum { STATE_COLLECTING, STATE_NORMAL, STATE_ALERT } sensor_state_t;
typedef enum { SQL_CONNECTED, SQL_LOST, SQL_DOWN } sql_state_t;

/* ---- Sensor ---- */
/* sensor_data_t: xem Mục 3.2 */
typedef struct {              /* một dòng của file room-sensor */
    uint16_t      room;
    sensor_id_t   id;
    sensor_type_t type;
} sensor_info_t;

/* ---- sbuffer ---- */
typedef struct sbuffer_node {
    sensor_data_t        data;
    bool                 read_by_data;      /* Data manager đã đọc    */
    bool                 read_by_storage;   /* Storage manager đã đọc */
    struct sbuffer_node *next;
} sbuffer_node_t;

typedef struct {
    sbuffer_node_t *head, *tail;
    size_t          size;
    bool            shutdown;              /* báo kết thúc           */
    pthread_mutex_t lock;
    pthread_cond_t  not_empty;             /* đánh thức thread đọc   */
} sbuffer_t;

/* ---- Connection manager ---- */
typedef struct {
    int                        fd;
    const protocol_adapter_t *adapter;
    uint8_t                    rx[RX_BUF_SIZE];  /* bộ đệm nhận riêng    */
    size_t                     rx_len;
    bool                       identified;       /* đã biết sensor ID    */
    sensor_id_t                sensor_id;
} connection_t;

typedef struct {
    int          listen_fd[4];                   /* cổng P .. P+3        */
    connection_t conns[MAX_CONN];
    size_t       n_conns;
} connmgr_t;

/* ---- Data manager ---- */
typedef struct {
    sensor_info_t  info;
    double         window[RUN_AVG_LENGTH];      /* vòng đệm giá trị trung binh */
    size_t         count, next;
    double         running_avg;
    sensor_state_t state;
} datamgr_sensor_t;

typedef struct { datamgr_sensor_t sensors[MAX_SENSORS]; size_t n; } datamgr_t;
typedef void (*sensor_handler_t)(datamgr_t *, datamgr_sensor_t *, const
sensor_data_t *);

/* ---- Storage manager + SQLite ---- */
typedef struct {
    sqlite3      *db;
    sqlite3_stmt *insert;
    sql_state_t   state;
    int           attempts;
    const char   *db_path;
} storagemgr_t;
/* SQLite: bảng SensorData(id, type, value, ts), xem Mục 3.6 */
```

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

```
/* ---- Shared status (shared memory) ---- */
typedef struct {
    sensor_id_t id;  uint16_t room;  sensor_type_t type;
    double      last_value, running_avg;  bool has_avg;
    sensor_state_t state;  bool connected;  time_t last_update;
} sensor_status_t;

typedef struct {
    pthread_mutex_t lock;                       /* PROCESS_SHARED + ROBUST */
    uint64_t        received, processed, stored;
    uint32_t        sbuffer_size;
    sql_state_t     sql_state;  uint8_t sql_retries;
    time_t          start_time;
    uint16_t        sensor_count;
    sensor_status_t sensors[MAX_SENSORS];
} gateway_status_t;

/* ---- Ngữ cảnh gốc của main process ---- */
typedef struct {
    uint16_t        base_port, http_port;
    sbuffer_t       sbuf;
    connmgr_t       conn;
    datamgr_t       data;
    storagemgr_t    stor;
    gateway_status_t *status;                   /* NULL cho tới Step 8   */
    pid_t           log_pid, web_pid;
    volatile sig_atomic_t stop;
} gateway_t;
```

4. Các Step triển khai

Các Step được thực hiện theo thứ tự: mỗi Step dựa trên kết quả của Step trước và có tiêu chí kiểm chứng riêng. Chỉ chuyển sang Step kế tiếp khi Step hiện tại đã đạt tiêu chí. Mỗi Step gồm mục tiêu, sơ đồ thể hiện vị trí của Step trong kiến trúc, nhiệm vụ, lựa chọn kỹ thuật cùng luận cứ, và kiểm chứng. Quy ước màu trên sơ đồ: thành phần đang triển khai ở Step hiện tại có viền xanh và nền xanh; thành phần đã hoàn thành ở các Step trước có viền đen và nền xám; thành phần chưa triển khai được làm mờ. Main process là khung chứa nên có nền xanh ở Step 1 (khi được tạo cùng ba thread rỗng) và ở Step 7 (khi các thành phần bên trong được tích hợp). Các Step được chia thành hai giai đoạn: giữa kỳ (Step 0–7) và cuối kỳ (Step 8–12).

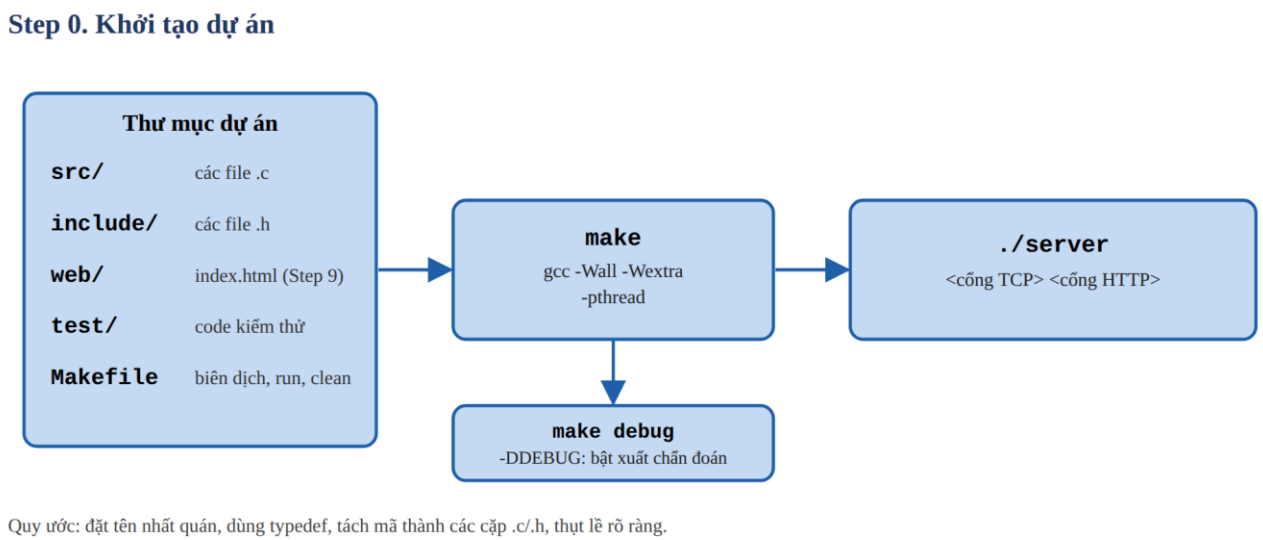
4.1 Giai đoạn 1 – Giữa kỳ: lõi gateway (Step 0–7)

Mục tiêu giai đoạn: hoàn thành lõi gateway chạy ổn định, chưa có giao diện web: nhận dữ liệu từ năm loại sensor qua bốn giao thức, xử lý theo loại, lưu vào SQLite, ghi nhật ký qua log process và kết thúc có kiểm soát. Phạm vi: Step 0–7. Hoàn thành khi phần Kiểm chứng của Step 0–7 đạt.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 0. Khởi tạo dự án

Mục tiêu: Có bộ khung dự án biên dịch được trước khi hiện thực logic.



Sơ đồ Step 0: cấu trúc dự án và quy trình biên dịch

Nhiệm vụ

- Tạo cấu trúc thư mục: src/, include/ (hoặc header đặt cạnh source), web/, test/ và Makefile.
- Thống nhất quy ước đặt tên, dùng typedef cho các kiểu dữ liệu, tách mã nguồn thành các cặp .c/.h và thụt lề nhất quán.
- Bổ sung chế độ debug (xuất thông tin chẩn đoán) phục vụ kiểm thử.

Lựa chọn kỹ thuật

- **Makefile với gcc -Wall -Wextra -pthread:** quy trình build lặp lại được; cảnh báo của trình biên dịch giúp phát hiện lỗi sớm; sinh viên phải biết biên dịch và chạy chương trình bằng một makefile đơn giản khi bảo vệ.
- **Chế độ debug bật bằng macro tiền xử lý (-DDEBUG):** bật/tắt thông tin chẩn đoán mà không phải sửa mã nguồn.

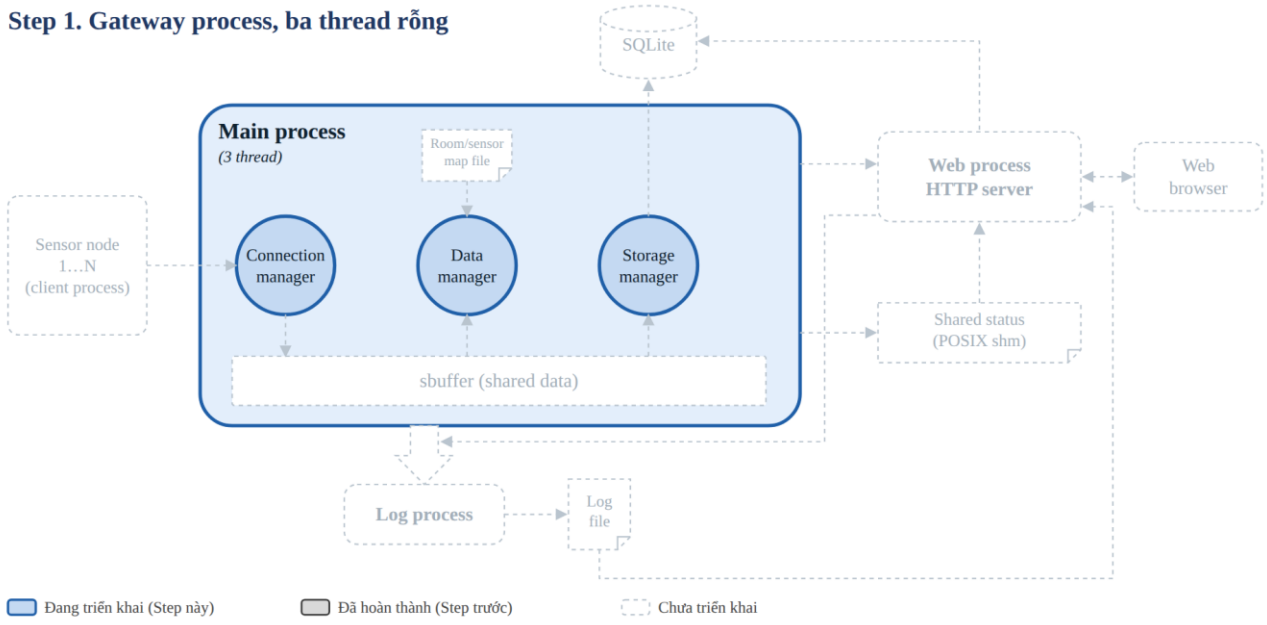
Kiểm chứng: lệnh make hoàn tất với main.c rỗng, không có cảnh báo.

- Ghi thử: main process ghi “Gateway started …” khi khởi động và “Gateway shutting down …” khi kết thúc; mỗi thread (Connection manager, Data manager, Storage manager) ghi một dòng cho biết thread đã khởi động.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 1. Sensor gateway process và ba thread rỗng

Mục tiêu: Có tiến trình sensor_gateway với ba thread (Connection manager, Data manager, Storage manager) chạy và kết thúc đúng, chưa có logic xử lý.



Sơ đồ Step 1: main process với ba thread rỗng

Nhiệm vụ

- Tạo chương trình sensor_gateway (./server). Đọc hai tham số dòng lệnh: cổng gốc P và cổng HTTP; báo lỗi và thoát nếu thiếu hoặc không hợp lệ.
- Khai báo ngữ cảnh gateway_t (Mục 3.9) chứa cờ kết thúc và các thành phần sẽ được bổ sung ở các Step sau.
- Tạo ba thread bằng pthread_create: connmgr, datamgr, storagemgr. Thân mỗi thread chỉ in thông tin chẩn đoán (chế độ debug) và chờ cờ kết thúc.
- Xử lý SIGINT và SIGTERM bằng cờ kết thúc; main thread chờ cờ này, báo các thread dừng rồi gọi pthread_join cho từng thread.
- Chưa dùng dữ liệu dùng chung ở Step này.

Lựa chọn kỹ thuật

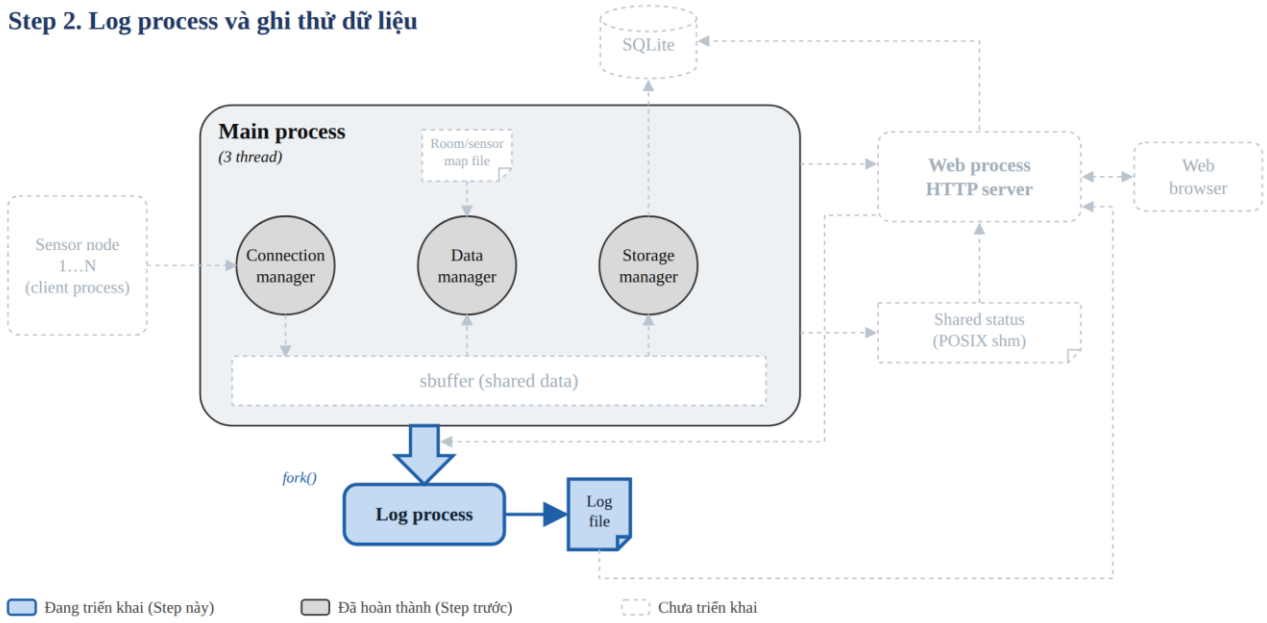
- **Ba thread trong một process:** ba thành phần sẽ dùng chung sbuffer ở Step 3; thread dùng chung không gian địa chỉ nên không phải sao chép dữ liệu giữa các thành phần.
- **Dựng khung trước, hiện thực logic sau:** vòng đời thread (tạo, dừng, join) được kiểm tra độc lập với lỗi logic xử lý dữ liệu, nên lỗi ở các Step sau dễ khoanh vùng.
- **Trình xử lý tín hiệu chỉ đặt cờ:** chỉ các hàm async-signal-safe mới được gọi trong trình xử lý tín hiệu, nên chỉ gán cờ volatile sig_atomic_t; việc dọn dẹp thực hiện ở main thread.
- **pthread_join:** thu hồi thread và tránh rò rỉ tài nguyên.

Kiểm chứng: chạy ./server rồi nhấn Ctrl+C: ba thread thoát, main kết thúc; valgrind --leak-check=full không báo rò rỉ; valgrind --tool=helgrind không có cảnh báo.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 2. Log process và ghi thử dữ liệu

Mục tiêu: Ghi nhật ký tập trung mà không làm chậm các thread xử lý.



Sơ đồ Step 2: log process, FIFO và gateway.log

Nhiệm vụ

- Main process dùng fork() tạo log process (process con).
- Dùng FIFO tên logFifo. Nếu FIFO chưa tồn tại khi main process hoặc log process khởi động, một trong hai process tạo nó bằng mkfifo().
- Mọi thread của main process (và web process ở Step 9) có thể gửi log-event; việc ghi vào FIFO từ nhiều thread phải thread-safe.
- Log-event là thông điệp ASCII. Với mỗi log-event nhận được, log process ghi một dòng <số thứ tự> <thời điểm> <nguồn> <thông điệp> vào gateway.log, theo định dạng và danh mục ở Mục 3.8.

Lựa chọn kỹ thuật

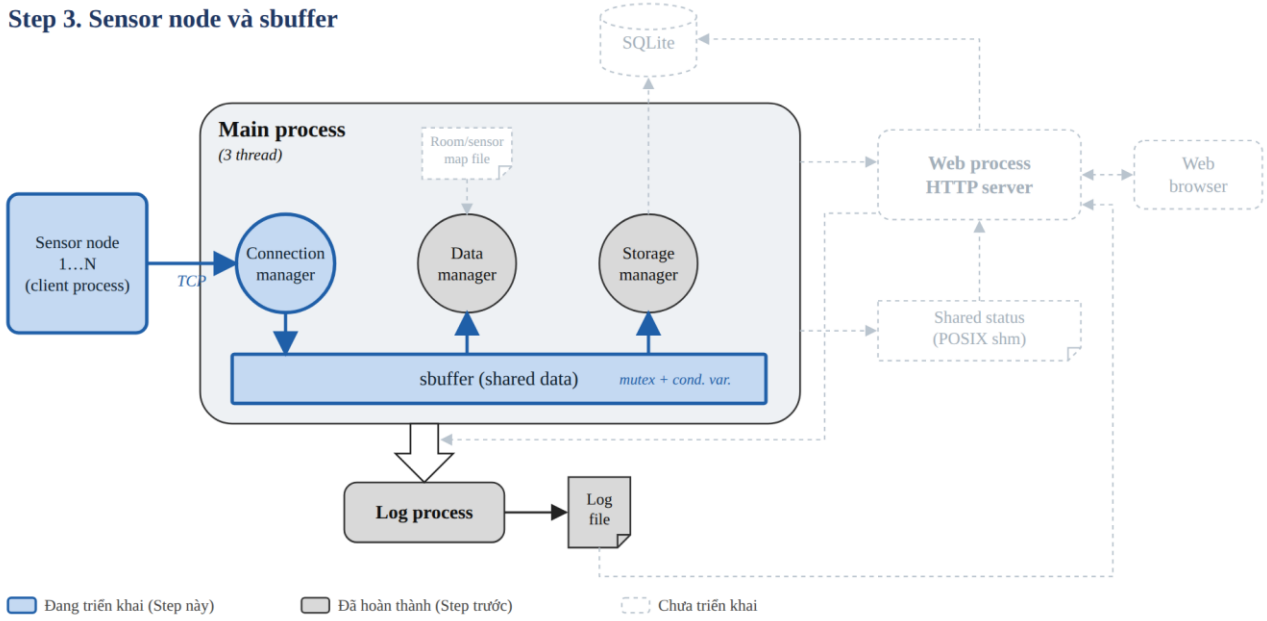
- **Log là process riêng:** ghi file là thao tác I/O chậm và độc lập với xử lý dữ liệu. Tách riêng giúp mỗi thread chỉ tốn một lệnh write() vào FIFO. Log process là nơi duy nhất ghi gateway.log nên số thứ tự được đảm bảo liên tục; lỗi trong log process không làm hỏng dữ liệu cảm biến.
- **FIFO (named pipe):** kênh một chiều, bảo toàn thứ tự; log process bị chặn trong read() khi chưa có dữ liệu nên không tiêu tốn CPU. Thông điệp được ghi bằng một lệnh write() không vượt quá PIPE_BUF (4096 byte trên Linux) là nguyên tử (atomic), nên nhiều bên ghi đồng thời không làm các thông điệp xen lẫn nhau. FIFO có tên giúp hai process không phụ thuộc vào thứ tự khởi động.
- **Mutex riêng cho hàm ghi log:** đảm bảo thread-safe và không lồng với khóa của sbuffer, do đó không tạo ra vòng chờ khóa.

Kiểm chứng: nhiều thread ghi log đồng thời; số thứ tự liên tục; không có dòng log nào bị cắt hoặc trộn lẫn; gateway.log có đủ dòng khởi động của MAIN, CONN, DATA, STOR theo định dạng ở Mục 3.8.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 3. Sensor node mô phỏng, sbuffer và Connection manager

Mục tiêu: Đưa dữ liệu từ sensor node mô phỏng vào sbuffer và kiểm chứng đường đi sensor node → Connection manager → sbuffer → hai thread đọc.



Sơ đồ Step 3: sensor node, Connection manager và sbuffer

Nhiệm vụ

- Chỉ dùng một sbuffer duy nhất để chia sẻ phép đo giữa ba thread; giải pháp dùng nhiều buffer không được chấp nhận.
- Connection manager ghi; Data manager và Storage manager đọc. Mọi thao tác đọc, ghi và cập nhật phải thread-safe.
- Mỗi phần tử mang hai cờ: “Data manager đã đọc” và “Storage manager đã đọc”. Phần tử chỉ bị loại bỏ khi cả hai cờ được bật.
- Thread đọc phải bị chặn (block) khi chưa có dữ liệu; cần có cơ chế báo kết thúc để đánh thức các thread khi gateway dừng.
- Viết (hoặc dùng) chương trình sensor_node mô phỏng một sensor nhiệt độ: kết nối tới cổng P và gửi định kỳ frame TBP (Mục 3.4.1, 3.7).
- Connection manager: đọc cổng gốc P từ tham số dòng lệnh thứ nhất (ví dụ ./server 1234), lắng nghe cổng P, chấp nhận nhiều sensor node đồng thời (không giả định số lượng tối đa), giải mã frame TBP 18 byte từ bộ đệm nhận của từng kết nối và ghi sensor_data_t vào sbuffer.
- Data manager và Storage manager tạm thời chỉ đọc sbuffer, in sensor_data_t ra (chế độ debug) và bật cờ đã đọc của mình, để kiểm tra cơ chế hai cờ.
- Ghi các log-event sau:
  - A sensor node with <sensorNodeID> has opened a new connection (sensor ID chỉ được biết sau frame hợp lệ đầu tiên nên ghi log sau frame đầu).
  - The sensor node with <sensorNodeID> has closed the connection
  - Connection attempt failed on <protocol> port (<reason>)

Lựa chọn kỹ thuật

- **Thread + dữ liệu dùng chung:** ba thread xử lý cùng một luồng phép đo; dùng chung không gian địa chỉ nên không phải sao chép dữ liệu giữa các thành phần, đơn giản và chi phí thấp.

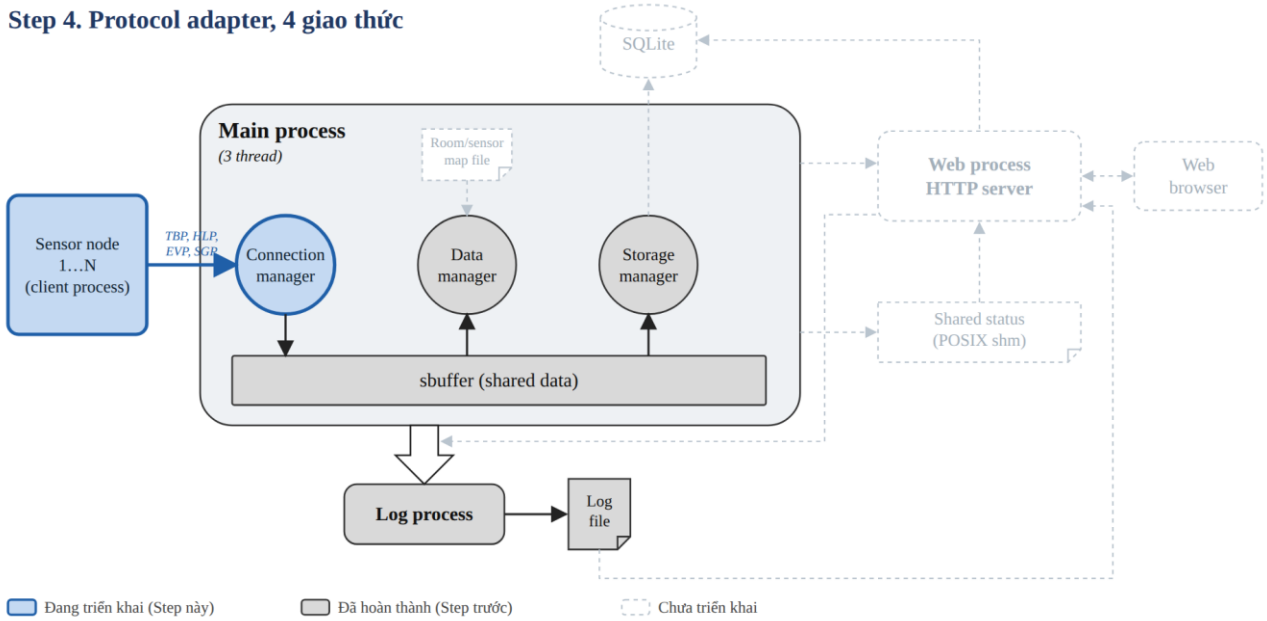
© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

- **Mutex + condition variable:** mutex bảo vệ cấu trúc dữ liệu; thread đọc chờ trên condition variable (trạng thái blocked, không tiêu tốn CPU) cho tới khi thread ghi báo hiệu có dữ liệu. Polling (kiểm tra lặp liên tục) lãng phí CPU, còn chạy tuần tự các thread làm mất ý nghĩa của đa luồng; cả hai phương án đều bị trừ điểm (bài lập trình tối đa 15/20).
- **Tránh deadlock:** mỗi thao tác chỉ giữ một khóa tại một thời điểm; không gọi hàm có thể chặn lâu (ghi cơ sở dữ liệu, ghi log, I/O mạng) khi đang giữ khóa của sbuffer.
- **Công bằng, tránh starvation:** vùng găng (critical section) rất ngắn, chỉ thao tác trên con trỏ và cờ, nên mọi thread đều truy cập được ngay khi khóa rảnh.
- **TCP socket cho mọi loại sensor:** sensor node là process hoặc thiết bị độc lập, có thể ở máy khác. TCP cung cấp luồng byte tin cậy, đúng thứ tự và cho biết khi đầu kia đóng kết nối (cần cho log). UDP có thể mất gói tin và không báo đóng kết nối; FIFO và shared memory chỉ dùng được trên cùng một máy.
- **Sensor node là client process riêng:** tách sensor node thành chương trình độc lập cho phép kiểm thử gateway như với thiết bị thật, chạy nhiều node cùng lúc và mô phỏng kết nối đến/đi bất kỳ lúc nào.
- **Phục vụ nhiều kết nối:** có thể dùng poll()/epoll để một thread phục vụ nhiều socket (kể cả socket lắng nghe), hoặc dùng thread phụ cho mỗi kết nối. Cần chọn một phương án và giải thích sự đánh đổi (số lượng thread, chi phí chuyển ngữ cảnh); việc ghi sbuffer vẫn phải tuân thủ cơ chế đồng bộ ở Step 3.

Kiểm chứng: chạy 3–5 sensor_node nhiệt độ cùng lúc với khoảng nghỉ ngắn; mỗi phép đo gửi đi xuất hiện đúng một lần trong đầu ra debug của Data manager và Storage manager (đúng sensor ID và giá trị); gateway.log có dòng mở/đóng kết nối; valgrind --tool=helgrind không phát hiện data race hay deadlock.

Step 4. Protocol adapter và các giao thức còn lại

Mục tiêu: Hỗ trợ đủ bốn giao thức và năm loại sensor: giải mã bằng protocol adapter, chuẩn hóa và đưa vào sbuffer.



Sơ đồ Step 4: Connection manager với bốn protocol adapter

Nhiệm vụ

- Mở rộng Connection manager từ một cổng (Step 3) lên bốn cổng TCP P, P+1, P+2 và P+3, mỗi cổng gắn với một protocol adapter (Mục 3.4, 3.5).

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

- Chấp nhận kết nối từ sensor node mới trên cả bốn cổng; phục vụ đồng thời nhiều sensor node, không giả định số lượng tối đa.
- Với mỗi kết nối, duy trì một bộ đệm nhận riêng. Sau mỗi lần recv(), gọi parse() của adapter tương ứng lặp lại cho đến khi trả PARSE_NEED_MORE. PARSE_OK: ghi sensor_data_t vào sbuffer. PARSE_ERROR: ghi log, bỏ số byte hỏng theo *consumed và tiếp tục; nêu rõ chính sách khi lỗi lặp lại (ví dụ đóng kết nối).
- Với giao thức có phản hồi (SGP): gửi ACK/NAK bằng build_reply().
- Ghi các log-event sau:
  - Received malformed frame on <protocol> port (<reason>)

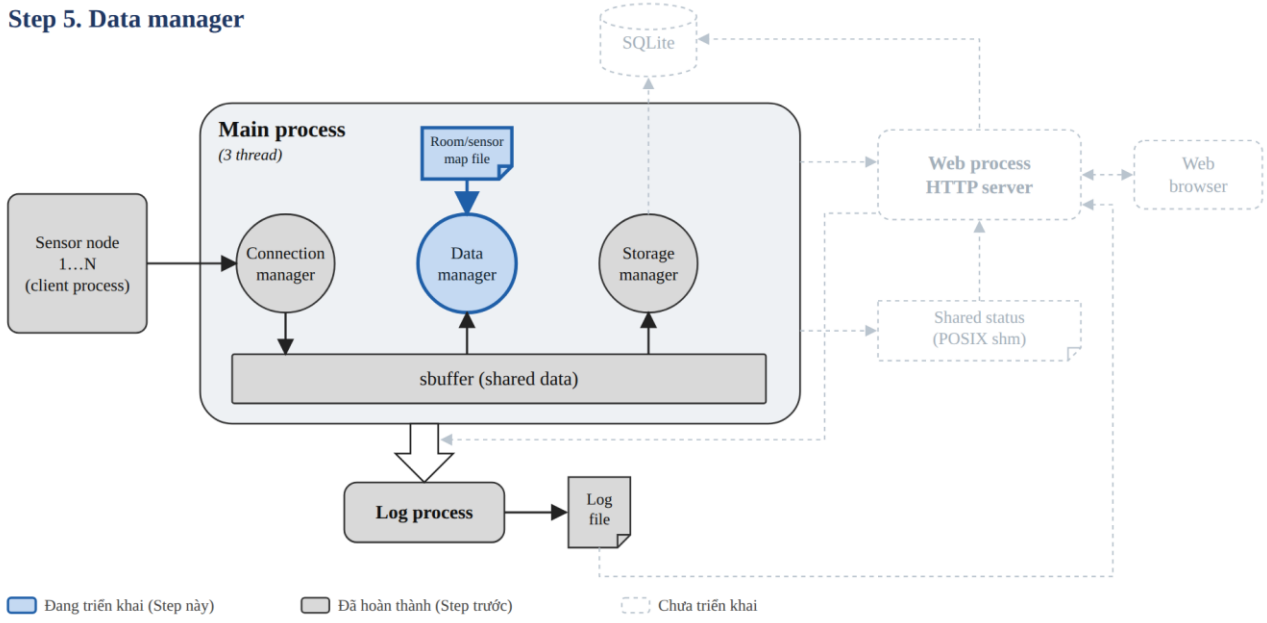
Lựa chọn kỹ thuật

- **Protocol adapter (bảng con trỏ hàm):** cô lập sự khác biệt giữa các giao thức vào một lớp riêng; từ sbuffer trở đi chỉ còn sensor_data_t. Thêm giao thức mới chỉ thêm một adapter, không sửa lõi. Đánh đổi là thêm một lớp gián tiếp, nhưng mỗi parse() có thể kiểm thử độc lập bằng các byte mẫu ở Mục 3.4.
- **Bộ đệm theo từng kết nối và PARSE_NEED_MORE:** TCP là luồng byte, không bảo toàn ranh giới thông điệp: một lần recv() có thể trả nửa frame hoặc nhiều frame. Gọi recv() một lần rồi giải mã ngay là sai; phải ghép đủ byte trước khi giải mã.
- **Giải mã từng trường, không ép con trỏ về struct:** tránh phụ thuộc vào padding, căn chỉnh và thứ tự byte của nền tảng. Dùng chuyển đổi little-endian (TBP) hoặc big-endian (EVP, SGP); với double, sao chép 8 byte vào uint64_t rồi memcpy sang double.
- **Mỗi giao thức một cổng:** cổng quyết định adapter nên không cần byte nhận dạng giao thức. Đánh đổi là phải mở nhiều socket lắng nghe; một cổng chung cần thêm bước nhận dạng giao thức.

Kiểm chứng: chạy 3–5 sensor mô phỏng cho mỗi loại, khoảng nghỉ ngắn; thử các tùy chọn --fragment, --glue, --corrupt; kiểm thử riêng từng parse() bằng các ví dụ ở Mục 3.4 (kể cả vector CRC 0x29B1).

Step 5. Data manager

Mục tiêu: Xử lý dữ liệu theo từng loại sensor và phát hiện bất thường.



Sơ đồ Step 5: Data manager đọc sbuffer và bảng room–sensor

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Nhiệm vụ

- Nạp bảng room–sensor (phòng, sensor ID, loại sensor) từ file định dạng ở Mục 3.6 khi khởi động. Bảng này là danh mục các sensor hợp lệ, dùng để xác thực sensor ID, loại sensor và gắn số phòng cho từng sensor.
- Đọc sensor_data_t từ sbuffer; kiểm tra sensor ID có trong bảng và loại sensor khớp với bảng.
- Xử lý theo loại sensor thông qua một bảng handler (Mục 3.3): giá trị liên tục (nhiệt độ, độ ẩm) dùng giá trị trung binh 5 phép đo rồi so ngưỡng; khói/gas so ngưỡng tức thời; chuyển động và cửa ghi nhận sự kiện hoặc thay đổi trạng thái.
- Không ghi giá trị giá trị trung binh vào dữ liệu dùng chung sbuffer; chỉ dùng nội bộ để ra quyết định (ngoại lệ dành cho shared status được nêu ở Step 8).
- Ghi log khi sensor chuyển sang trạng thái cảnh báo (không lặp lại ở mỗi phép đo). Các log-event:
    - The sensor node with <sensorNodeID> reports it’s too cold (running avg temperature = <value>)
    - The sensor node with <sensorNodeID> reports it’s too hot (running avg temperature = <value>)
    - The sensor node with <sensorNodeID> reports it’s too dry (running avg humidity = <value>)
    - The sensor node with <sensorNodeID> reports it’s too humid (running avg humidity = <value>)
    - The sensor node with <sensorNodeID> detected motion
    - The sensor node with <sensorNodeID> reports the door is open / closed
    - The sensor node with <sensorNodeID> reports a smoke/gas alarm (concentration = <value> ppm)
    - Received sensor data with invalid sensor node ID <node-ID> (cũng dùng khi loại sensor không khớp bảng room–sensor)

Lựa chọn kỹ thuật

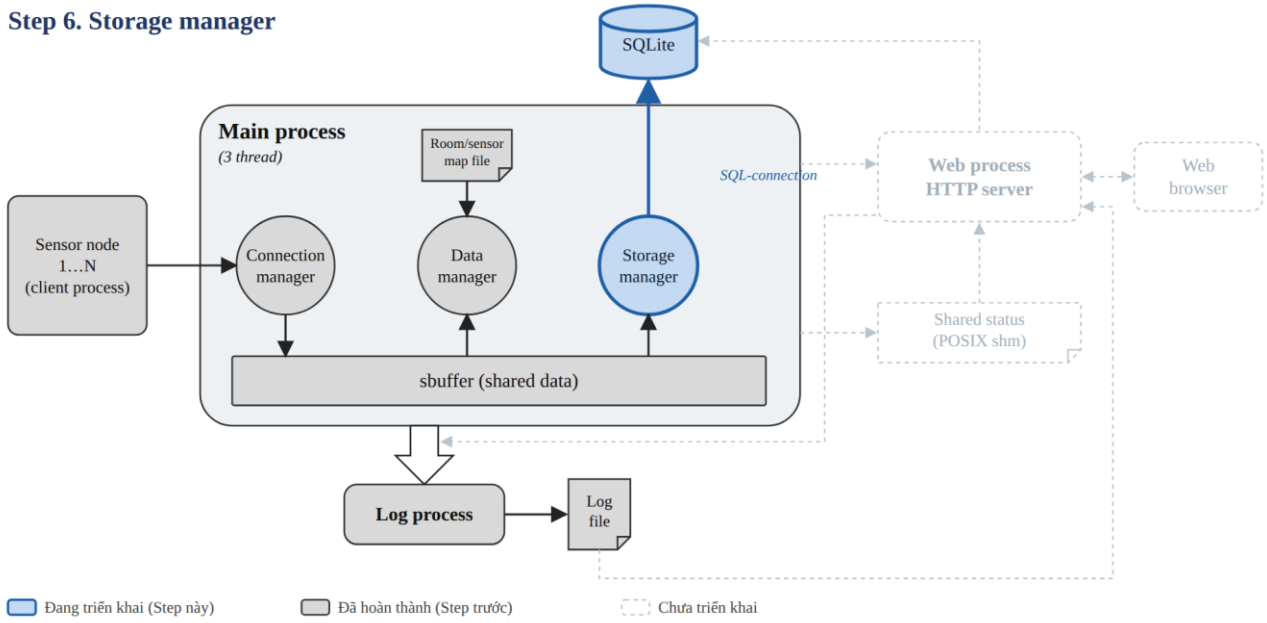
- **Thread riêng:** phân tích dữ liệu độc lập với I/O mạng (Connection manager) và I/O đĩa (Storage manager). Cơ sở dữ liệu chậm không làm đình trệ việc nhận dữ liệu, và ngược lại.
- **Bảng handler theo loại sensor:** mỗi loại một hàm xử lý với cùng chữ ký, chọn qua bảng tra theo sensor_type_t. Thêm loại mới chỉ thêm một dòng và một hàm, thay vì sửa các chuỗi if/switch rải rác.
- **Khói/gas so ngưỡng tức thời, không lấy trung bình:** giá trị trung binh làm chậm phản ứng, trong khi độ trễ của cảnh báo an toàn quan trọng hơn việc lọc nhiễu. Đánh đổi là dễ báo động giả; có thể giảm bằng điều kiện vượt ngưỡng nhiều mẫu liên tiếp (tùy chọn). Nhiệt độ và độ ẩm biến đổi chậm và có nhiễu, nên giá trị trung binh phù hợp.
- **Chuyển động và cửa là sự kiện, không phải chuỗi đo:** không có khái niệm trung bình; chỉ cần ghi nhận thời điểm và trạng thái.
- **Trạng thái giá trị trung binh là cục bộ của thread này,** nên không cần khóa; thread chỉ chạm tới sbuffer khi đọc phần tử và bật cờ.

Kiểm chứng: dùng dữ liệu giả lập có ngưỡng đã biết, mỗi loại sensor ít nhất một ca vượt ngưỡng và một ca bình thường; thử sensor ID lạ và loại không khớp bảng; đối chiếu kết quả với log.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 6. Storage manager

Mục tiêu: Lưu phép đo vào SQLite và xử lý tình huống mất kết nối cơ sở dữ liệu.



Sơ đồ Step 6: Storage manager ghi vào SQLite

Nhiệm vụ

- Đọc sensor_data_t từ sbuffer và ghi (INSERT) vào bảng SensorData của SQLite (lược đồ ở Mục 3.6); tạo bảng khi thiết lập kết nối.
- Nếu kết nối tới cơ sở dữ liệu thất bại: chờ một khoảng thời gian rồi thử lại. Các phép đo được giữ trong sbuffer cho tới khi kết nối khôi phục. Nếu sau 3 lần thử vẫn thất bại, gateway kết thúc.
- Ghi các log-event sau:
  - Connection to SQL server established.
  - New table <name-of-table> created.
  - Connection to SQL server lost.
  - Unable to connect to SQL server.

Lựa chọn kỹ thuật

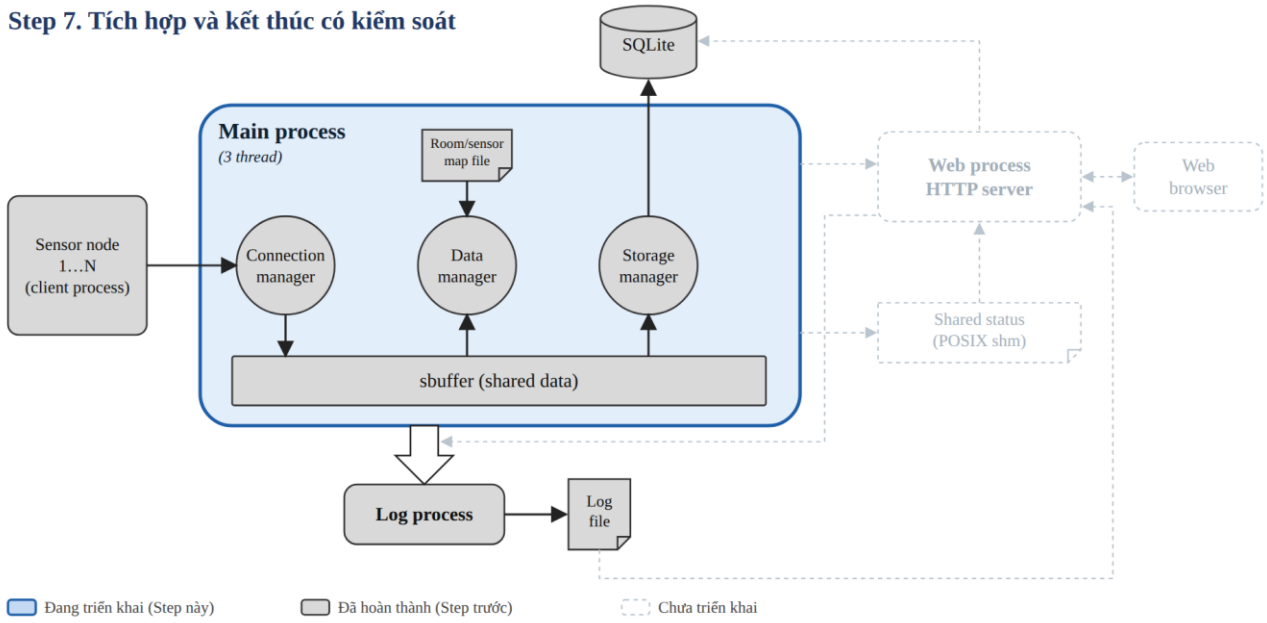
- **SQLite:** cơ sở dữ liệu nhúng dạng file, không cần server riêng; một file có thể được nhiều process mở (web process đọc).
- **Một bảng cho mọi loại sensor:** nhờ dữ liệu đã chuẩn hóa thành sensor_data_t, một bảng với cột type là đủ; truy vấn lịch sử theo sensor ID không phụ thuộc loại. Đánh đổi: cột value dùng chung kiểu REAL cho cả sự kiện 0/1, chấp nhận được ở quy mô này; tách bảng theo loại làm tăng số bảng và mã mà không có lợi ích tương ứng.
- **Thread riêng:** thao tác ghi đĩa chậm không được chặn việc nhận và phân tích dữ liệu.
- **Giữ phép đo trong sbuffer khi cơ sở dữ liệu lỗi:** sbuffer đóng vai trò bộ đệm nên không mất dữ liệu. Đánh đổi là bộ nhớ tăng khi sự cố kéo dài, vì vậy số lần thử lại bị giới hạn ở 3.

Kiểm chứng: đổi tên hoặc khóa file cơ sở dữ liệu giữa chừng; quan sát gateway thử lại và kết thúc đúng yêu cầu.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 7. Tích hợp và kết thúc có kiểm soát

Mục tiêu: Ba thread và log process hoạt động cùng nhau; gateway kết thúc mà không để lại tài nguyên.



Sơ đồ Step 7: tích hợp lõi gateway và quy trình kết thúc

Nhiệm vụ

- Chạy Connection manager, Data manager và Storage manager cùng log process.
- Khi kết thúc: báo các thread dừng, pthread_join từng thread, đóng socket, FIFO và cơ sở dữ liệu, wait() process con, giải phóng bộ nhớ.

Lựa chọn kỹ thuật

- pthread_join để thu hồi thread và tránh rò rỉ tài nguyên; wait() để tránh process zombie. Cờ kết thúc kèm việc đánh thức condition variable giúp các thread đang chờ thoát khỏi trạng thái blocked.

Kiểm chứng: valgrind --leak-check=full không báo rò rỉ; chạy liên tục vài phút với nhiều sensor.

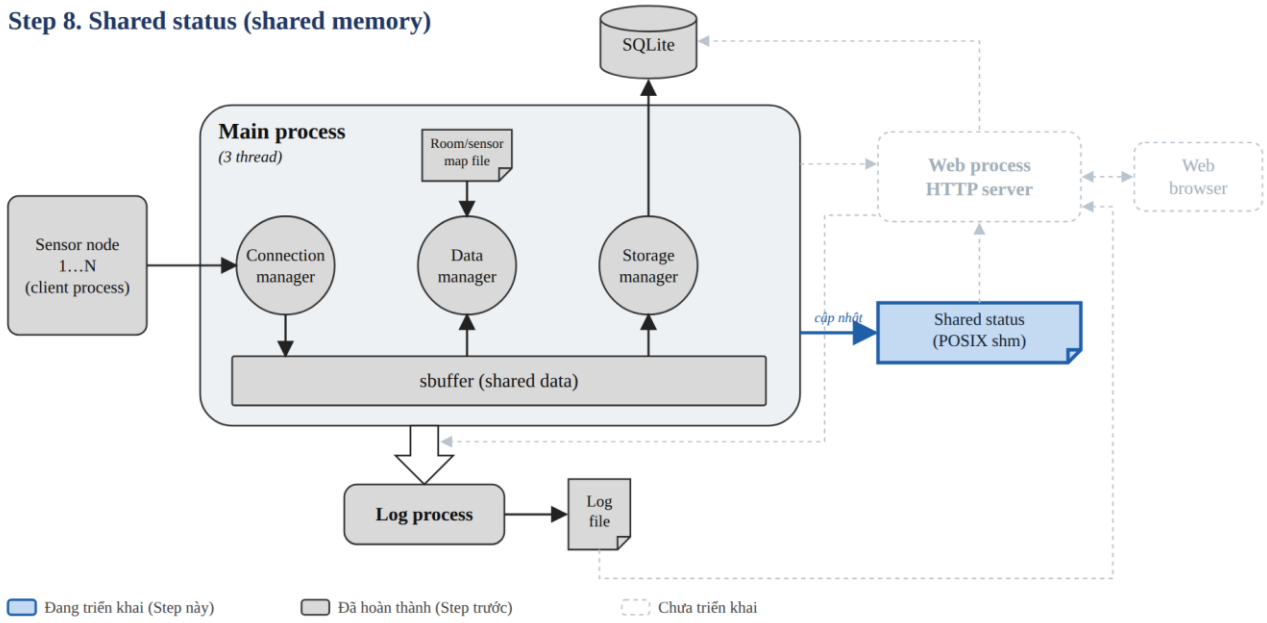
4.2 Giai đoạn 2 – Cuối kỳ: giao diện web và hoàn thiện (Step 8–12)

Mục tiêu giai đoạn: bổ sung giao diện web chạy trên web process riêng, không ảnh hưởng đến lõi gateway; sau đó kiểm thử tổng thể và nộp bài. Phạm vi: Step 8–12. Hoàn thành khi phần Kiểm chứng của Step 8–12 đạt.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 8. Shared status (shared memory)

Mục tiêu: Cho phép web process, vốn có không gian địa chỉ riêng, đọc được trạng thái hiện hành của gateway.



Sơ đồ Step 8: shared status giữa gateway và web process

Nhiệm vụ

- Main process tạo vùng shared memory POSIX (shm_open, ftruncate, mmap) trước khi fork web process. Vùng này chứa: bộ đếm (số phép đo đã nhận, đã xử lý, đã lưu); bảng sensor (phòng, ID, loại sensor, giá trị mới nhất, giá trị trung binh nếu có, trạng thái, tình trạng kết nối, thời điểm cập nhật gần nhất); trạng thái kết nối SQL (đã kết nối, mất kết nối, số lần thử lại); số phần tử trong sbuffer; thời điểm khởi động.
- Ba thread cập nhật vùng này. Data manager được ghi giá trị trung binh mới nhất vào đây chỉ để hiển thị (ngoại lệ của Step 5, vì shared status không phải là sbuffer).
- Bảo vệ vùng này bằng mutex liên process (PTHREAD_PROCESS_SHARED) có thuộc tính robust.
- Main process gọi shm_unlink khi kết thúc.

Lựa chọn kỹ thuật

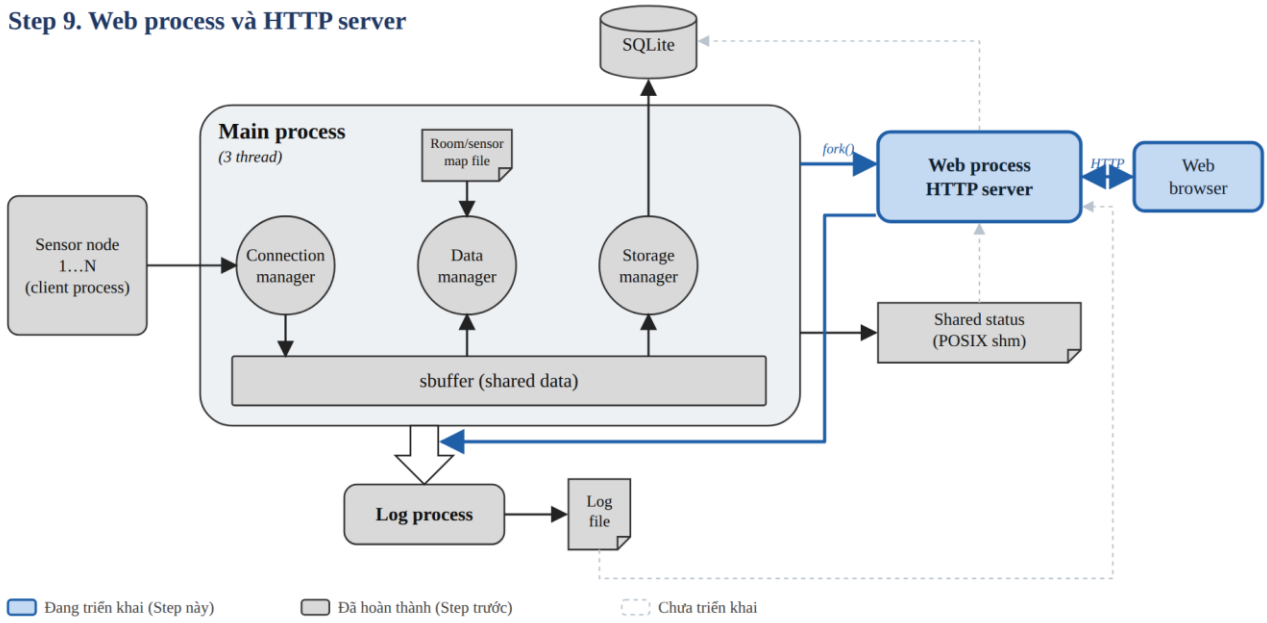
- **Lý do cần IPC:** các process không dùng chung bộ nhớ như các thread, nên dữ liệu phải đi qua một cơ chế IPC.
- **Shared memory:** phù hợp nhất với dữ liệu nhỏ, thay đổi liên tục và được đọc nhiều lần; đọc chỉ là truy cập bộ nhớ, không cần tuần tự hóa (serialization) hay giao thức. Pipe hoặc socket buộc gateway phải chủ động đẩy dữ liệu; file trạng thái bị ghi liên tục gây tốn I/O và có thể bị đọc phải dữ liệu chưa nhất quán (torn read).
- **Mutex PROCESS_SHARED + robust:** mutex mặc định chỉ có hiệu lực trong phạm vi một process. Thuộc tính robust cần thiết vì web process có thể bị kết thúc khi đang giữ khóa; khi đó thread nhận mã EOWNERDEAD và gọi pthread_mutex_consistent() để khôi phục.

Kiểm chứng: một chương trình đọc độc lập hiển thị số liệu thay đổi theo thời gian thực.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 9. Web process và HTTP server

Mục tiêu: Xây dựng HTTP server độc lập phục vụ trình duyệt mà không ảnh hưởng đến lõi gateway.



Sơ đồ Step 9: web process và HTTP server

Nhiệm vụ

- Main process dùng fork() tạo process thứ ba (web process), tương tự log process.
- Đọc cổng HTTP từ tham số dòng lệnh thứ hai (./server 1234 8080); mặc định chỉ lắng nghe trên 127.0.0.1, với vòng lặp accept() riêng. Hiện thực bằng socket; nếu dùng thư viện web server bên ngoài thì phải nêu rõ và trích dẫn nguồn.
- Phục vụ đồng thời nhiều trình duyệt.
- Phân tích yêu cầu HTTP/1.1: GET / trả về trang HTML; /api/status, /api/log?n= và /api/history?id=&n= trả về JSON. Đường dẫn không tồn tại trả mã 404, phương thức khác GET trả mã 405; yêu cầu sai định dạng không được làm server ngừng hoạt động.
- Ghi các log-event sau (gửi qua logFifo, mỗi thông điệp một lệnh write() không vượt quá PIPE_BUF):
  - Web server started on port <port>.
  - Web server received an invalid request (status code <status>).

Lựa chọn kỹ thuật

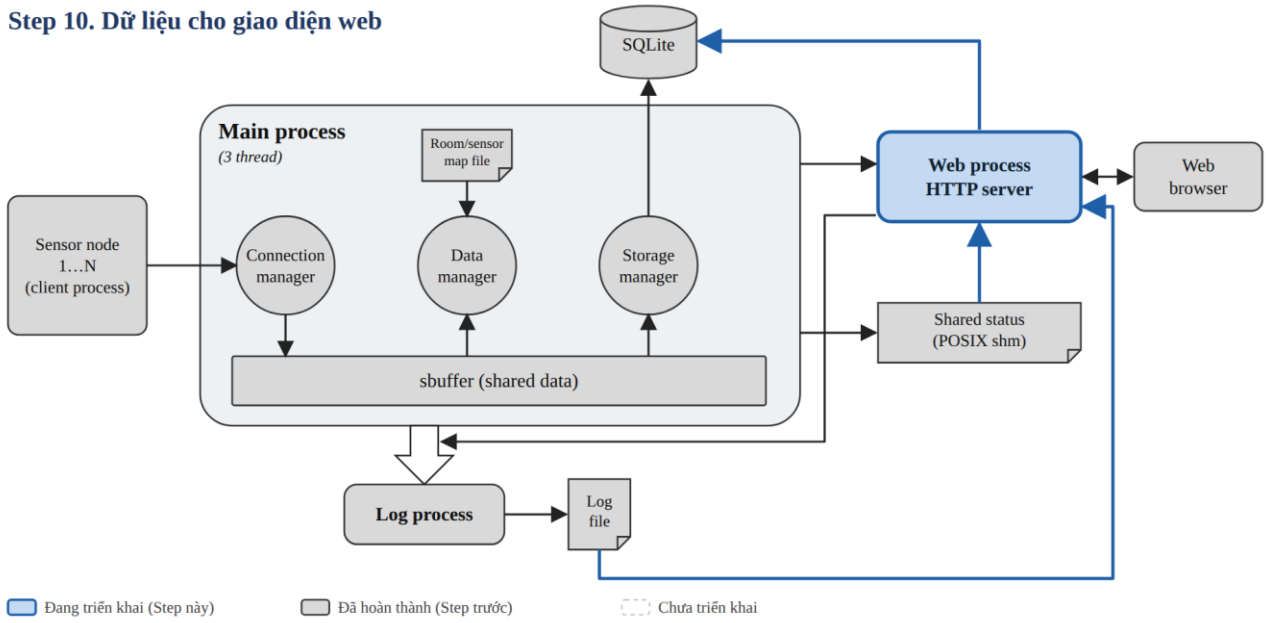
- **Web là process riêng, không phải thread thứ tư:** bộ phân tích HTTP tiếp xúc trực tiếp với bên ngoài nên là điểm dễ phát sinh lỗi nhất. Nếu là thread, một lỗi bộ nhớ trong đó có thể làm sập cả main process; nếu là process, gateway vẫn tiếp tục thu nhận và lưu phép đo. Các thao tác I/O chặn (accept, read) cũng không làm chậm ba thread, và web process có thể dừng hoặc khởi động lại độc lập.
- **Đánh đổi:** không dùng chung bộ nhớ nên cần IPC (Step 8, 10) và phải quản lý vòng đời process (Step 11).
- **HTTP trên TCP:** là giao thức chuẩn mà trình duyệt hỗ trợ sẵn. Polling đơn giản hơn WebSocket hay Server-Sent Events và đủ đáp ứng yêu cầu cập nhật mỗi vài giây.

Kiểm chứng: gọi các endpoint bằng curl; mở đồng thời hai tab trình duyệt.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 10. Cung cấp dữ liệu cho giao diện web

Mục tiêu: Giao diện hiển thị đúng, tự làm mới và chỉ có quyền đọc (read-only).

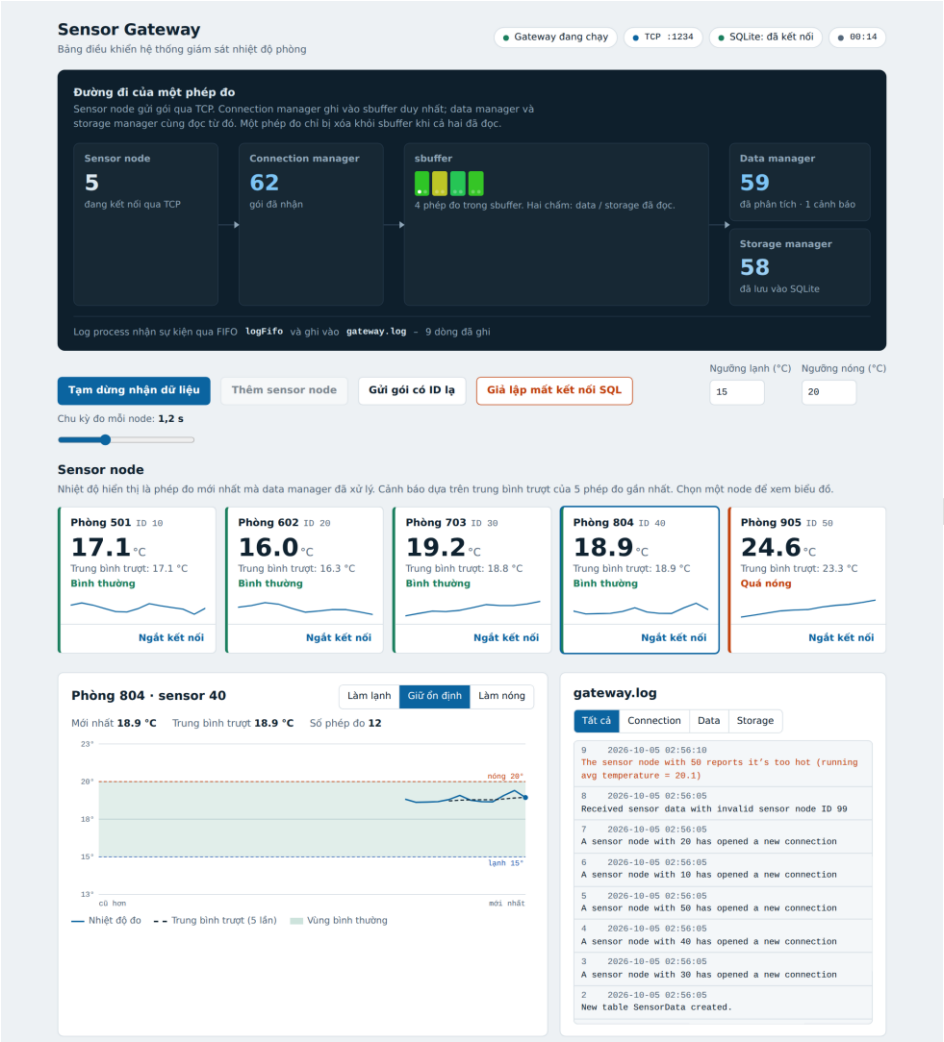


Sơ đồ Step 10: ba nguồn dữ liệu của giao diện web

Nhiệm vụ

- /api/status: sao chép nhanh dữ liệu từ shared memory (giữ mutex trong thời gian ngắn) rồi trả về JSON. Với mỗi sensor node đã từng kết nối: số phòng, sensor ID, loại sensor, giá trị mới nhất (kèm đơn vị), trạng thái (đang thu thập / bình thường / cảnh báo theo loại sensor) và tình trạng kết nối. Với gateway: trạng thái kết nối SQL, tổng số phép đo đã nhận và đã lưu, số phần tử trong sbuffer, thời gian hoạt động.
- /api/log?n=: n dòng cuối của gateway.log (mặc định 50), dòng mới nhất ở trên, cho phép lọc theo nguồn (connection, data, storage, web). Đọc từ file gateway.log, không đọc từ FIFO.
- /api/history?id=&n=: n bản ghi gần nhất của một sensor, truy vấn từ SQLite mở ở chế độ chỉ đọc; xử lý trường hợp cơ sở dữ liệu đang bị khóa (SQLITE_BUSY). Với giá trị liên tục (nhiệt độ, độ ẩm, khói/gas) trả chuỗi giá trị để vẽ biểu đồ; với chuyển động và cửa trả danh sách sự kiện theo thời gian.
- web/index.html: một file duy nhất (HTML, CSS, JavaScript), tự làm mới ít nhất mỗi 2 giây (polling), không cần tải lại trang. Hình 4 là bố cục tham khảo (bản minh họa chỉ hiển thị cảm biến nhiệt độ; giao diện đầy đủ hiển thị theo từng loại sensor).

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.



Hình 4: Giao diện web tham khảo (luồng dữ liệu, trạng thái sensor, biểu đồ và gateway.log)

Lựa chọn kỹ thuật

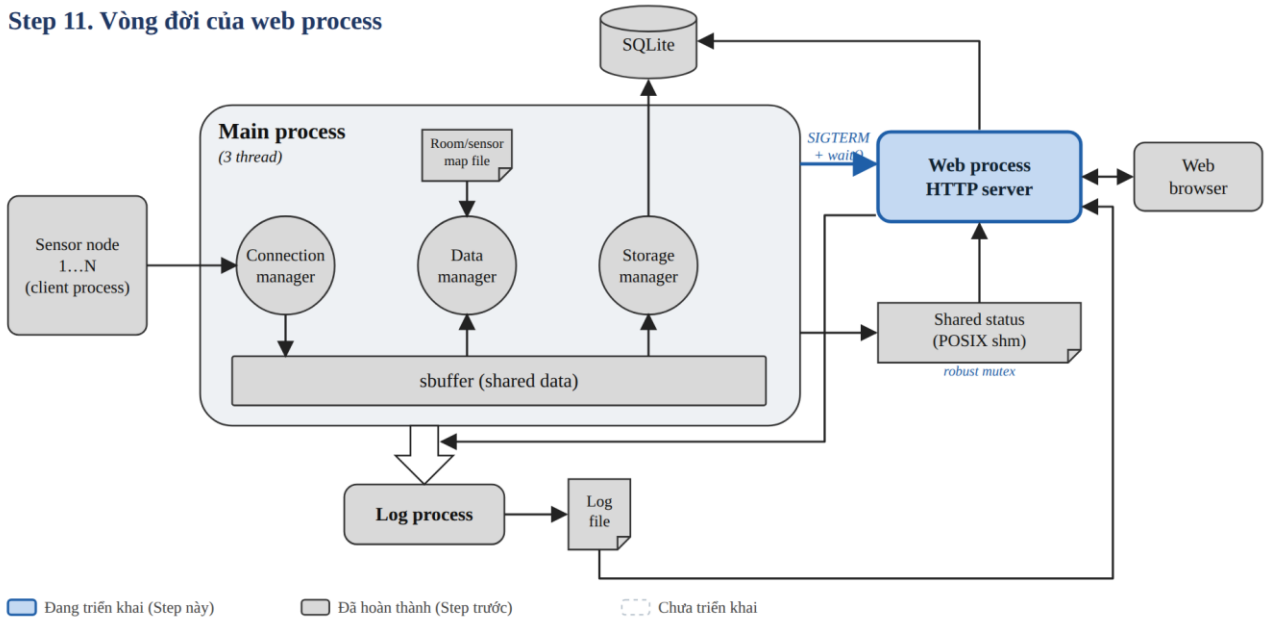
- **Mỗi loại dữ liệu lấy từ nguồn phù hợp nhất:** trạng thái tức thời từ shared memory; lịch sử từ SQLite (đã có sẵn, cơ sở dữ liệu tự quản lý khóa, mở chỉ đọc nên web không thể sửa dữ liệu); nhật ký từ file. Nếu xảy ra tranh chấp khóa giữa thread ghi và web process, có thể bật chế độ WAL (journal_mode=WAL) để reader không bị writer chặn.
- **Không đọc FIFO:** FIFO chỉ có một bên đọc là log process; nếu web process đọc sẽ lấy mất thông điệp của log process.
- **Sao chép rồi nhả khóa:** web process chỉ giữ mutex đủ lâu để chép dữ liệu ra bản cục bộ, nên một trình duyệt chậm không làm chậm các thread của gateway.

Kiểm chứng: sensor chuyển sang trạng thái cảnh báo được hiển thị (mỗi loại sensor ít nhất một ca); trạng thái mất kết nối SQL được hiển thị; nhật ký hiển thị đúng.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 11. Vòng đời và xử lý lỗi của web process

Mục tiêu: Web process không được trở thành nguồn gây lỗi cho gateway.



Sơ đồ Step 11: vòng đời và xử lý lỗi của web process

Nhiệm vụ

- Khi gateway kết thúc (kể cả sau 3 lần kết nối SQL thất bại ở Step 6): main process kết thúc web process và gọi wait() để không để lại zombie.
- Nếu web process gặp lỗi và dừng, gateway vẫn tiếp tục hoạt động.
- Nếu web process bị kết thúc khi đang giữ mutex, các thread vẫn lấy được khóa (xử lý EOWNERDEAD).
- Xử lý trường hợp trình duyệt đóng kết nối giữa chừng (SIGPIPE hoặc lỗi write).

Lựa chọn kỹ thuật

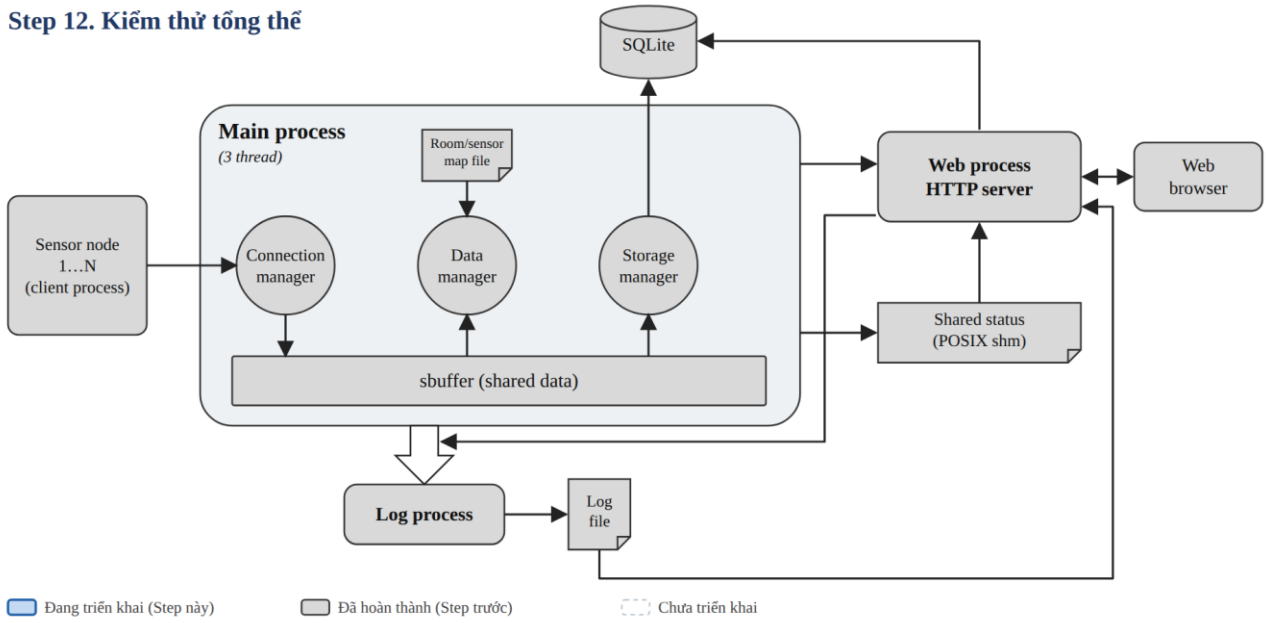
- Đây là chi phí đánh đổi (trade-off) của việc tách process: lõi gateway phải tiếp tục hoạt động khi web process lỗi, và web process phải được dọn dẹp khi gateway kết thúc. Robust mutex, wait() và xử lý tín hiệu là các công cụ để đạt được điều đó.

Kiểm chứng: kill -9 web process: ps không còn zombie, gateway không bị treo.

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

Step 12. Kiểm thử tổng thể và nộp bài

Mục tiêu: Chương trình ổn định, đã được kiểm chứng và sẵn sàng để bảo vệ.



Sơ đồ Step 12: kiểm thử toàn hệ thống

Nhiệm vụ

- Chạy bộ kiểm thử: đủ năm loại sensor chạy đồng thời, khoảng nghỉ ngắn, frame phân mảnh/dính nhau/hỏng (--fragment, --glue, --corrupt), mất kết nối SQL, sensor ID không hợp lệ, ngắt kết nối đột ngột, kết thúc gateway khi web đang mở.
- Chạy valgrind và helgrind lần cuối; loại bỏ mọi cảnh báo của trình biên dịch.
- Viết tài liệu tự đánh giá mã nguồn (code self-review).
- Tải bài lên labtools.groept.be trước hạn; bài phải biên dịch và chạy được.
- Đọc lại Mục 5 và luyện trả lời các câu hỏi bảo vệ dựa trên chính mã nguồn của mình.

5. Câu hỏi bảo vệ gợi ý

1. Vì sao Connection manager, Data manager và Storage manager là thread, còn log và web là process?
2. Vì sao log dùng FIFO mà không ghi trực tiếp vào gateway.log từ mỗi thread?
3. PIPE_BUF liên quan thế nào tới việc nhiều bên cùng ghi vào FIFO?
4. Vì sao trạng thái cho web dùng shared memory mà không dùng pipe hoặc file?
5. Vì sao không dùng polling cho sbuffer? Condition variable giải quyết vấn đề gì?
6. Khi hai thread cùng đọc một phần tử, khi nào phần tử được loại bỏ và vì sao dùng cờ?
7. Khi kết nối SQL bị mất, dữ liệu đi đâu? Vì sao gateway kết thúc sau 3 lần thử?
8. Nếu web process bị kill -9, gateway còn hoạt động đúng không? Chứng minh bằng cách nào?
9. Vì sao web đọc nhật ký từ file mà không đọc từ FIFO?
10. Vì sao dùng protocol adapter? Thêm một loại sensor mới cần sửa những chỗ nào?
11. TCP là luồng byte: nếu một frame đến thành hai lần recv() thì điều gì xảy ra và bạn xử lý ra sao?
12. Vì sao khói/gas dùng CRC và ACK/NAK còn nhiệt độ thì không?
13. Vì sao khói/gas so ngưỡng tức thời mà nhiệt độ dùng giá trị trung binh?

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.

6. Sản phẩm nộp và tiêu chí chấp nhận

6.1 Tiêu chí chung cho cả hai giai đoạn

- Mã nguồn có cấu trúc và dễ đọc: tách file source và header hợp lý, đặt tên biến và hàm rõ nghĩa, dùng typedef, thụt lề đúng, không viết ‘spaghetti code’.
- File README.md của dự án (giới thiệu dự án, mục tiêu và hướng dẫn chạy)
- Bằng chứng cho các tiêu chí kiểm chứng của Step tương ứng, như nhật ký chạy thử, đầu ra valgrind và helgrind, ảnh chụp màn hình. Mỗi kết quả ghi rõ lệnh đã chạy.
- Video định dạng mp4, có giọng nói giải thích trực tiếp trong video

DevLinux

© DevLinux Co., Ltd. All rights reserved. This document is a proprietary product of DevLinux and, as such, any unauthorized use, disclosure, or reproduction of this publication or portions thereof in any form, without written permission from DevLinux, is strictly prohibited.